



JUnits

🕒 Created	December 23, 2024 11:26 PM
🕒 Last edited time	January 14, 2025 8:02 PM

Unit testing with JUnit

this is for dependency junit5-4.13.1

Test cases are written for testing code

```
@Test
public void test(){
    fail("fail");
}
```

the @Test is imported from org.junit.Test, fail is a method that fails the test.

```
@Test
public void test(){
    assertEquals("hello", "hello");
    // 1st is accepted and 2nd is result
}
```

```
//assertsEquals does literal value comparision, that is even if the
//object content is same the object created will have different
ids. hence will not match
//and test case will fail.
```

Test cases conditions:

1. Test cases name according to method being tested or condition being tested eg testAddAttachment()
2. all test should be public void

```
import org.junit.Assert;
Assert.assertFalse("message for failure", false);
Assert.assertTrue("message for failure", true); (this message will be printed on the
console)
```

Test the boolean conditions.

@Before used to run something before running the test cases.

@After used to run something before running the test cases.

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;

public class Main {
    public static void main(String[] args) throws IOException {

        System.out.println("give");
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        String[] array = br.readLine().split("");
        Main testmethod = new Main();
        String result = testmethod.testMethod(array);
        System.out.println(result);
    }
}
```

```

    }

    public String testMethod(String[] array){
        int counter = 0;
        for (int i = 0; i < array.length; i++) {
            if (array[i].equals(array[array.length - 1 -
i])) {
                counter++;
            }
        }
        if (counter == array.length){
            return ("YES");
        } else {
            return ("NO");
        }
    }
}

```

```

import org.junit.*;

public class MainTest {

    Main palindrom = new Main();

    @BeforeClass
    public static void oneTimeSetUp(){
        System.out.println("before class");
    }

    @AfterClass
    public static void oneTimeEnding(){
        System.out.println("before class");
    }

    @Before
    public void setup(){
        System.out.println("give");
    }
}

```

```

@After
public void afterTest(){
    System.out.println("out");
}

@Test
public void testPalindromSuccess(){
    String test = "ADDA";
    String[] array = test.split("");
    String result = palindrom.testMethod(array);
    Assert.assertEquals("YES", result);
}

@Test
public void testPalindromFailure(){
    String test = "ADGHDA";
    String[] array = test.split("");
    String result = palindrom.testMethod(array);
    Assert.assertEquals("NO", result);
}
}

```

```

before class
give
out
give
out
after class

```

@BeforeClass runs only once at the start of the test class and **@AfterClass** works only at the very end of the test class

comparing arrays

```

@Test
public void arrayTest(){
    int[] num1 = {23, 56, 6, 8};
    int[] num2 = {6, 8, 23, 56};
}

```

```

        Arrays.sort(num1);
        Assert.assertArrayEquals(num2, num1);
    }
    // test will pass

```

Exception testing

```

@Test(expected = NullPointerException.class)
public void exceptionException(){
    int[] num = null;
    Arrays.sort(num);
} // will pass

@Test(expected = NullPointerException.class)
public void exceptionExceptionfailed(){
    int[] num = {};
    Arrays.sort(num);
} // test will fail as {} is different from null

```

performance test

```

@Test(timeout = 100)
public void performaceTest(){
    int[] array = {12, 56, 7, 8, 34};
    for (int i=0; i<=100000; i++){
        array[0] = i;
        array[2] = (i+2);
        Arrays.sort(array);
    }
} // will pass if total time taken is under 100ms.

```

parameterized test

@Parameters is annotated to a method that returns a collections and in the method mention array with multiple inputs and their respected outputs

Steps to Write Test Case for user input and output.

To write a test case for the code that involves `System.out.println` and reading from `System.in` (like the example you provided), you'll need to handle both of these elements in the test. Here's how to approach it:

1. **Redirecting `System.out`**: You can use a `ByteArrayOutputStream` to capture the output printed by `System.out.println`.
2. **Mocking `System.in`**: To simulate user input for `BufferedReader`, you can use `System.setIn()` to provide an `InputStream` with predefined data (such as a simulated user input).
3. **Handling `BufferedReader` and Input**: The `BufferedReader` reads from the `System.in` stream, so you must ensure the input stream is set up correctly to simulate user input during the test.

Example Code to Test

Example Code

```
import java.io.*;

public class MainClass {
    public static void main(String[] args) throws IOException {
        System.out.println("give");

        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        String[] array = br.readLine().split("");

        // Output the content of the array (as a simple check)
        for (String s : array) {
            System.out.print(s);
        }
    }
}
```

```
}  
}
```

Writing the Test Case Using JUnit

Here's how you can write a JUnit test for this code, where you'll capture the output and mock the user input.

Test Code Using JUnit

```
import org.junit.jupiter.api.Test;  
import java.io.*;  
  
import static org.junit.jupiter.api.Assertions.*;  
  
public class MainClassTest {  
  
    @Test  
    public void testMainWithInput() throws IOException {  
        // Simulate the input from System.in  
        String simulatedInput = "abc";  
        InputStream inputStream = new ByteArrayInputStream  
(simulatedInput.getBytes());  
        System.setIn(inputStream);  
  
        // Redirect System.out to capture the printed output  
        ByteArrayOutputStream outContent = new ByteArrayOutputStream();  
        System.setOut(new PrintStream(outContent));  
  
        // Run the main method  
        MainClass.main(new String[0]);  
  
        // Verify the output printed to the console  
        assertTrue(outContent.toString().contains("give"));  
  
        // Verify that the input was processed correctly  
        assertTrue(outContent.toString().contains("a"));  
    }  
}
```

```
        assertTrue(outContent.toString().contains("b"));
        assertTrue(outContent.toString().contains("c"));
    }
}
```

Explanation of the Test

1. Simulating User Input:

- We simulate input to `System.in` using `ByteArrayInputStream`. The string `"abc"` is provided as the simulated input, which will be read by the `BufferedReader` in your code.
- `System.setIn(inputStream)` is used to redirect `System.in` to read from the `ByteArrayInputStream` instead of the actual console.

2. Capturing Output:

- To capture what is printed to `System.out`, we redirect it using `ByteArrayOutputStream` and `System.setOut(new PrintStream(outContent))`. This allows us to read the content that would have been printed to the console.

3. Assertions:

- We use `assertTrue` to check that the expected output is present in the captured output.
 - First, we check that the string `"give"` is printed (as it is printed first by the `System.out.println`).
 - Then, we check that each of the characters from the input string `"abc"` is printed one by one. The `split("")` method splits the input string into an array of characters, so we expect each character to be printed individually.

Key Points to Handle in the Test

- **Mocking `System.in`**: The main challenge is to provide simulated input for `BufferedReader.readLine()`. This is done by creating a `ByteArrayInputStream` with the input data.
- **Capturing `System.out`**: You need to redirect `System.out` to capture printed text (using `ByteArrayOutputStream`).

- **Edge Cases:** You can also write additional tests for different inputs or cases where the input might be empty, null, or invalid.

Additional Considerations

- **System.in Closing:** If the code involves closing `System.in` or performing cleanup, you might need to handle that as well, but in most cases for unit testing, the above setup should suffice.
- **Simulating Different Inputs:** You can simulate various inputs (e.g., empty string, long strings) by changing the input string in the `ByteArrayInputStream`.

Conclusion

By redirecting `System.in` and `System.out`, you can simulate both the input and output in your tests, allowing you to verify the behavior of code that interacts with the console. This approach helps ensure that your program behaves as expected under different input scenarios.

Performance Metrics

Writing performance test cases for Java code involves measuring the system's efficiency, including how well it performs under varying loads or resource utilization. Performance tests typically focus on key aspects like execution time, memory usage, response time, scalability, and throughput.

Here are the key steps and guidelines for writing performance test cases in Java:

1. Define the Performance Metrics

Before writing test cases, define the key performance metrics you want to measure. Common metrics include:

- **Execution time:** How long the code takes to execute.
- **Throughput:** The amount of work done in a given period.
- **Latency:** The delay before the code starts producing results.
- **Memory Usage:** The amount of memory consumed by the code during execution.
- **Concurrency and Scalability:** How well the system performs when multiple users or processes access it simultaneously.

2. Use Profiling Tools

You can use profiling tools to measure the performance of your code. Some popular tools in the Java ecosystem include:

- **JMH (Java Microbenchmarking Harness):** JMH is a tool specifically designed for benchmarking Java code and measuring microperformance accurately.
- **VisualVM or JProfiler:** These tools allow you to monitor and analyze memory usage, CPU usage, and threads in real-time.
- **Apache JMeter:** Useful for load and stress testing, simulating multiple users or systems interacting with your code.

3. Writing JMH Performance Test Cases

If you're focusing on precise benchmarking of small code snippets (like functions or methods), JMH is the recommended approach. JMH provides high precision when measuring performance by eliminating JVM warm-up time and other noise factors.

steps:

Write a Benchmark Test with JMH: Here's a basic example of a JMH test case:

```
import org.openjdk.jmh.annotations.Benchmark;
import org.openjdk.jmh.annotations.State;
import org.openjdk.jmh.annotations.Scope;

@State(Scope.Thread) // Defines the scope of the benchmark
                        (one instance per thread)
public class PerformanceTest {
    // Sample method to benchmark
    public int sum(int a, int b) {
        return a + b;
    }
    // Benchmark method
    @Benchmark
    public void testSum() {
        sum(10, 20); // The method you want to benchmark
    }
}
```

```
}  
}
```

Run the Benchmark:

Use the JMH command to run the benchmark from your terminal or IDE:

```
mvn clean install  
java -jar target/benchmarks.jar
```

JMH will provide you with results, such as the average time taken for method executions, the throughput, etc.

4. Writing Load or Stress Test Cases

For larger-scale performance tests (such as load and stress tests), consider tools like Apache JMeter or Gatling to simulate multiple users or systems interacting with your codebase.

Example with JMeter:

1. Create a Test Plan:

- Define a **Thread Group** (simulate multiple users).
- Add a **Sampler** (e.g., HTTP Request or JDBC Request).
- Add **Listeners** to collect and visualize results (e.g., Graph Results, View Results Tree).

2. Execute Load Test:

- Execute your test plan with different loads (e.g., 100, 1000, 10,000 threads).
- Collect metrics like response time, error rates, and throughput.

5. Measure Memory Usage with JProfiler or VisualVM

These tools are useful for measuring memory consumption and detecting memory leaks.

1. Using VisualVM:

- Start your Java application with the `Dcom.sun.management.jmxremote` flag.
- Use VisualVM to connect to your running Java process.

- Monitor memory usage, CPU usage, and thread activity.

2. Using JProfiler:

- Attach JProfiler to your running application.
- Analyze memory usage, garbage collection, CPU profiling, and threading.

Mockito

Mocking is a process of developing the objects that act as the **mock** or **clone** of the real objects. In other words, mocking is a testing technique where mock objects are used instead of real objects for testing purposes. Mock objects provide a specific (dummy) output for a particular (dummy) input passed to it.

1. Introduction to Mockito

Mockito is a mocking framework that allows you to create mock objects for testing purposes. It helps in isolating the unit of work by mocking dependencies, making it easier to test the behavior of the code.

2. Key Concepts

- **Mock**: A mock object simulates the behavior of a real object. It is used to verify interactions and ensure the correct behavior of the unit under test.
- **Stub**: A stub provides predefined responses to method calls. It is used for state verification.
- **Fake**: A fake object has a working implementation but is not suitable for production. It is used for testing purposes.

3. Basic Annotations

- `@Mock` : Creates a mock object.
- `@InjectMocks` : Injects mock objects into the tested class.
- `@Spy` : Creates a spy object, which is a partial mock.
- `@Captor` : Captures argument values for further assertions.
- The test class should be `@RunWith(MockitoJUnitRunner.class)`

- `@Mock` creates a mock. `@InjectMocks` creates an instance of the class and injects the mocks that are created with the `@Mock` (or `@Spy`) annotations into this instance. Note you must use `@RunWith(MockitoJUnitRunner.class)` or `Mockito.initMocks(this)` to initialize these mocks and inject them (JUnit 4). With JUnit 5, you must use `@ExtendWith(MockitoExtension.class)`.

4. Creating Mocks

You can create mocks using the `Mockito.mock()` method or annotations:

```
// Using Mockito.mock()
List mockedList = Mockito.mock(List.class);

// Using @Mock annotation
@Mock
List mockedList;
```

5. Stubbing Methods

Stubbing allows you to define the behavior of mock methods:

```
when(mockedList.get(0)).thenReturn("first element");
```

Note: error =

`org.mockito.exceptions.misusing.MissingMethodInvocationException:`
`when()` requires an argument which has to be 'a method call on a mock'. →
 this happens when

Mockito's `when()` method is used incorrectly. Specifically, it happens if you attempt to mock or stub a method that hasn't been properly mocked or if the method invocation isn't actually happening on a mock object.

6. Verifying Interactions

Mockito provides methods to verify interactions with mock objects:

```
verify(mockedList).add("one"); //mockedList is class add i
s method
verify(mockedList, never()).add("one"); //to see if method
is never called
```

```
verify(mockedList, times(1)).add("one"); // method called only once
```

```
then(mockedList).should(never()).add("one"); //same as verify, but with then
```

7. Advanced Features

- **Argument Matchers:** Use `any()`, `eq()`, etc., to match arguments.
- **Exception Handling:** Mock methods to throw exceptions using `thenThrow()`.
- **Spying:** Create partial mocks to test real methods while stubbing others.

8. ArgumentCaptor

In Mockito, the `ArgumentCaptor` is used to capture arguments passed to mock methods during testing. It allows you to inspect or assert on the arguments that were passed into a mocked method call.

```
public class ArgumentCaptorExample {
    private List<String> mockedList;
    @Before
    public void setUp() {
        mockedList = mock(List.class);
    }
    @Test
    public void testArgumentCaptor() {
        // Create an ArgumentCaptor for String type
        ArgumentCaptor<String> captor = ArgumentCaptor.forClass(String.class);

        // Perform method call on the mocked object
        mockedList.add("Hello");
        mockedList.add("World");

        // Capture the arguments that were passed to the add method
        verify(mockedList, times(2)).add(captor.capture());
    }
}
```

```

        // Retrieve the captured arguments
        List<String> capturedArguments = captor.getAllValues();

        // Assertions on captured values
        assertEquals(2, capturedArguments.size());
        assertEquals("Hello", capturedArguments.get(0));
        assertEquals("World", capturedArguments.get(1));
    }
}

```

8. Integration with JUnit

Mockito integrates seamlessly with JUnit for unit testing:

```

@RunWith(MockitoJUnitRunner.class)
public class ExampleTest {
    @Mock
    List mockedList;

    @Test
    public void test() {
        when(mockedList.get(0)).thenReturn("first element");
        assertEquals("first element", mockedList.get(0));
    }
}

```

9. Best Practices

- Keep tests simple and focused.
- Use mocks only when necessary.
- Avoid overusing spies as they can make tests harder to understand.

MockitoAnnotations.openMocks(this);

The `@BeforeEach` annotation in JUnit 5 is used to mark a method that should be executed **before** each test method runs. It's commonly used to set up the test

environment, initialize objects, or mock dependencies that will be shared across multiple test methods.

What is happening here:

```
@BeforeEach
public void setUp() {
    MockitoAnnotations.openMocks(this);
}
```

- `@BeforeEach` : This tells JUnit to run the `setUp()` method **before each test**. This is similar to `@Before` in JUnit 4, but in JUnit 5, it's renamed to `@BeforeEach` to make it more descriptive.
- `MockitoAnnotations.openMocks(this)` : This initializes the **Mockito annotations** in the current test class (`this`). Mockito uses annotations such as `@Mock` , `@InjectMocks` , etc., to manage test doubles (mock objects). `openMocks(this)` is responsible for setting up these mock objects and injecting them into the class under test (`artifactService` in the example).

In simple terms, `MockitoAnnotations.openMocks(this)` ensures that your mocked dependencies (e.g., `@Mock` fields) are properly initialized so you can use them in your tests. Without it, your `@Mock` fields would not be set up, and Mockito wouldn't be able to mock their behavior.

Detailed Breakdown:

- `@Mock` : Marks a field to be a mock. It allows you to define behavior for that mock in your tests.

```
@Mock
private Response response;
```

- `@InjectMocks` : Tells Mockito to create an instance of the class under test and inject all the mocks into it.

```
@InjectMocks
private ArtifactService artifactService;
```


- `openMocks(this)` : Initializes all the mock objects and injects them into the class under test. Without this call, the mocks would not be created and injected, and you would likely encounter a `NullPointerException` .

Example Without `@BeforeEach` Setup:

If you were not using `@BeforeEach` or `MockitoAnnotations.openMocks(this)` , you would need to manually initialize each mock object for every test, which can lead to duplication and increased code complexity.

PowerMock

PowerMock is a Java testing framework that extends the functionality of other popular mocking frameworks such as Mockito and EasyMock. It provides additional capabilities to mock static methods, constructors, private methods, and more, which are difficult or impossible to mock using standard mocking frameworks.

Key features of PowerMock include:

1. **Mocking Static Methods:** You can mock static methods in classes, which is usually not possible with other mocking frameworks like Mockito.
2. **Mocking Constructors:** PowerMock can be used to mock new object creation (i.e., constructors).
3. **Mocking Private Methods:** It can mock private methods and make them testable, which is particularly useful for testing complex classes.
4. **Mocking Final Classes and Methods:** PowerMock can mock final classes and methods, which are otherwise not mockable in frameworks like Mockito.
5. **Mocking System Classes:** PowerMock can mock system classes, such as `System` , `Thread` , and others.

```
@RunWith(PowerMockRunner.class)
@PrepareForTest(SomeClassWithStaticMethod.class)
public class SomeTest {
    @Test
    public void testStaticMethod() {
        PowerMockito.mockStatic(SomeClassWithStaticMethod.c
```

```

lass);

        // Mock behavior of static method
        Mockito.when(SomeClassWithStaticMethod.staticMethod
        ()).thenReturn("Mocked!");

        // Assert the behavior
        String result = SomeClassWithStaticMethod.staticMet
        hod();
        Assert.assertEquals(result, "Mocked!");
    }
}

```

Setup Dependencies:

```

<dependency>
    <groupId>org.powermock</groupId>
    <artifactId>powermock-api-mockito2</artifactId>
    <version>2.0.9</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.powermock</groupId>
    <artifactId>powermock-module-junit4</artifactId>
    <version>2.0.9</version>
    <scope>test</scope>
</dependency>

```

example for private methods:

```

@Test
public void test(){
    List<Integer> stats = Arrays.asList(1, 2, 3);
    when(class.getstats()).thenReturn(stats);
    long result = Whitebox.invokeMethod(systemUnderTest, "p
    rivateMethod");
}

```

```
    assertEquals(6, result);  
}
```